

DEPLOYMENT GUIDE

Auto Parts CRM

AWS deployment process using Amazon RDS PostgreSQL

Use this guide when deploying: Follow the sections in order: create AWS database, prepare server, configure environment variables, run Prisma migrations, build both apps, start services, and configure Nginx/SSL.

Layer	Recommended AWS service	Purpose
Database	Amazon RDS for PostgreSQL	Managed PostgreSQL database for Prisma models.
Compute	EC2 Ubuntu	Runs NestJS backend and Next.js frontend Node servers.
Cache / Queue	ElastiCache Redis, optional	Redis cache and BullMQ support when enabled.
Proxy	Nginx on EC2	Routes frontend and API domains to local Node ports.
Process manager	PM2	Keeps backend and frontend alive after reboot/deploy.

Production ports: backend runs on **3000** and frontend runs on **3001** before Nginx maps them to public domains.

1. Create AWS RDS PostgreSQL

- 1 Open AWS Console, go to RDS > Databases > Create database.
- 2 Select PostgreSQL and choose a Dev/Test or Production template.
- 3 Use database name **auto_parts_crm**, then set a strong username and password.
- 4 Keep the DB inside your VPC. Prefer private access instead of public access.
- 5 Configure the DB security group to allow port **5432** only from the EC2 server security group.
- 6 Enable automated backups and copy the RDS endpoint after creation.

```
DATABASE_URL="postgresql://DB_USER:DB_PASSWORD@RDS_ENDPOINT:5432/auto_parts_crm?schema=public"
```

2. Prepare EC2 Ubuntu Server

```
sudo apt update
sudo apt install -y git nginx
curl -fsSL https://deb.nodesource.com/setup_22.x | sudo -E bash -
sudo apt install -y nodejs
sudo npm install -g pm2
```

```
git clone <your-repo-url>
cd auto-parts-crm
npm install --prefix backend
npm install --prefix frontend
```

3. Backend Environment

Create **backend/.env** with production values:

```
DATABASE_URL="postgresql://DB_USER:DB_PASSWORD@RDS_ENDPOINT:5432/auto_parts_crm?schema=public"
JWT_SECRET="use-a-long-secure-random-secret"
JWT_EXPIRES_IN_SECONDS=86400
REDIS_HOST="your-redis-host-or-127.0.0.1"
REDIS_PORT=6379
REDIS_USERNAME=""
REDIS_PASSWORD=""
REDIS_DB=0
REDIS_CACHE_ENABLED=true
REDIS_CACHE_KEY_PREFIX="auto-parts-crm:cache:"
REDIS_TLS_ENABLED=false
REDIS_CONNECT_TIMEOUT_MS=5000
ORDERS_CACHE_TTL_SECONDS=60
BULLMQ_PREFIX="auto-parts-crm"
```

Redis note: If Redis is not deployed yet, set `REDIS_CACHE_ENABLED=false`. If BullMQ workers are required, deploy Redis through AWS ElastiCache or another managed Redis service.

4. Frontend Environment

Create **frontend/.env.production**:

```
NEXT_PUBLIC_APP_NAME="Auto Parts CRM"
NEXT_PUBLIC_APP_URL="https://your-domain.com"
NEXT_PUBLIC_API_TIMEOUT_MS=10000
NEXT_PUBLIC_TOAST_DURATION_MS=5000
BACKEND_API_URL="https://api.your-domain.com"
BACKEND_API_TIMEOUT_MS=10000
AUTH_COOKIE_MAX_AGE_SECONDS=86400
```

5. Run Prisma Production Migration

Run production migrations against the RDS database. Use **migrate deploy**, not **migrate dev**, for deployment.

```
cd backend
npm run prisma:generate
npx prisma migrate deploy
cd ..
```

6. Build Backend And Frontend

On Linux EC2:

```
npm run build --prefix backend  
npm run build --prefix frontend
```

On Windows local machine:

```
npm.cmd run build --prefix backend  
npm.cmd run build --prefix frontend
```

7. Start Production Apps With PM2

```
pm2 start "npm run start:prod --prefix backend" --name auto-parts-backend
pm2 start "npm run start --prefix frontend" --name auto-parts-frontend
pm2 save
pm2 startup
```

8. Configure Nginx Reverse Proxy

```
server {
    server_name your-domain.com;
    location / {
        proxy_pass http://127.0.0.1:3001;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}

server {
    server_name api.your-domain.com;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```
sudo apt install -y certbot python3-certbot-nginx
sudo certbot --nginx
```

9. Deployment Runbook

Use this sequence for future deployments:

```
git pull
npm install --prefix backend
npm install --prefix frontend
npm run prisma:generate --prefix backend
cd backend && npx prisma migrate deploy && cd ..
npm run build --prefix backend
npm run build --prefix frontend
pm2 restart auto-parts-backend
pm2 restart auto-parts-frontend
```

10. Production Checklist

- RDS is private and port 5432 is not exposed publicly.
- EC2 security group allows HTTP/HTTPS and restricts SSH.

- Backend .env uses the RDS endpoint and a strong JWT_SECRET.
- Prisma migrate deploy has completed successfully.
- Backend build and frontend build both pass.
- PM2 shows both processes online.
- Nginx routes frontend and API domains correctly.
- SSL certificate is installed and auto-renewal is enabled.
- RDS automated backups are enabled.

References

- AWS RDS PostgreSQL: <https://aws.amazon.com/rds/postgresql/>
- AWS RDS create DB:
https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_CreateDBInstance.html
- AWS RDS security groups:
<https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.RDSSecurityGroups.html>
- Prisma production migrations:
<https://www.prisma.io/docs/orm/prisma-client/deployment/deploy-database-changes-with-prisma-migrate>
- Next.js deployment: <https://nextjs.org/docs/app/getting-started/deploying>